Experiment 10

Aim : Explore and implement Autoencoders for image denoising

Dataset : MNIST

Theory :

An Autoencoder is a type of Artificial Neural Network used to learn efficient data codings in an unsupervised manner. The specific goal of a **Denoising Autoencoder** is to take a "corrupted" or "noisy" version of an image and reconstruct the original, clean version.

1. The Architecture: Encoder and Decoder

- **The Encoder:** This part of the network compresses the input image into a compact, lower-dimensional representation called the **Latent Space** or "Bottleneck." It uses Convolutional layers (Conv2D) to extract features and Pooling layers (MaxPooling2D) to reduce the image size.
- **The Decoder:** The decoder takes the compressed representation and tries to rebuild the original image. It uses **UpSampling** layers to increase the image size back to its original dimensions (28×28).

2. How Denoising Works

Instead of the standard autoencoder task (Input -> Output), a Denoising Autoencoder is trained with:

- **Input:** An image with added random Gaussian noise.
- **Target:** The original clean image.
  By comparing the noisy input to the clean target, the network learns to ignore the random noise (the "garbage") and only keep the meaningful patterns that form the digits.

3. Key Components

- **Convolutional Layers:** These layers are better than standard dense layers for images because they preserve the spatial relationship between pixels.
- **Sigmoid Activation:** The final layer uses a Sigmoid function because the pixel values are normalized between 0 and 1. Sigmoid ensures the output pixels also fall within this specific range.
- **Loss Function (Binary Crossentropy):** Even though this is a regression-like task (predicting pixel values), Binary Crossentropy is often used for normalized image reconstruction as it treats pixel intensities as probabilities of being "on" or "off."

Dataset Theory (MNIST with Gaussian Noise):

The MNIST dataset consists of 28*28 grayscale images.

- **Gaussian Noise:** This is a statistical noise where the "graininess" added to the image follows a normal distribution.
- **Clipping:** After adding noise, pixel values might go below 0 or above 1. We use np.clip to force them back into the valid 0.0 to 1.0 range so the model can process them correctly.

Code and Outputs :

Step 1 : Load and Preprocess the dataset

```
import tensorflow as tf

# 1. Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

print(f"Original training images shape: {x_train.shape}")
print(f"Original test images shape: {x_test.shape}")

# 2. Normalize the image data
x_train_normalized = x_train.astype('float32') / 255.0
x_test_normalized = x_test.astype('float32') / 255.0

print(f"Normalized training images min: {x_train_normalized.min()}, max:
{x_train_normalized.max()}")
print(f"Normalized test images min: {x_test_normalized.min()}, max: {x_test_normalized.max()}")

# 3. Reshape the image data to include a channel dimension
x_train_processed = x_train_normalized[..., tf.newaxis]
x_test_processed = x_test_normalized[..., tf.newaxis]

# 4. Store the preprocessed training images in a variable named x_train_processed and the preprocessed
test images in x_test_processed.
# (Already done in the previous step, variable names match)

print(f"Processed training images shape: {x_train_processed.shape}")
print(f"Processed test images shape: {x_test_processed.shape}")
```

Output :

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Original training images shape: (60000, 28, 28)
Original test images shape: (10000, 28, 28)
Normalized training images min: 0.0, max: 1.0
Normalized test images min: 0.0, max: 1.0
Processed training images shape: (60000, 28, 28, 1)
Processed test images shape: (10000, 28, 28, 1)
```

Step 2 : Add noise to images

```
import numpy as np

# 1. Define a function to add Gaussian noise
def add_gaussian_noise(images, noise_factor=0.2):
    noise = noise_factor * np.random.normal(loc=0.0, scale=1.0, size=images.shape)
    noisy_images = images + noise
    noisy_images = np.clip(noisy_images, 0.0, 1.0)
    return noisy_images

# Define the noise factor
noise_factor = 0.2

# 2. Apply noise to x_train_processed
x_train_noisy = add_gaussian_noise(x_train_processed, noise_factor)

# 3. Apply noise to x_test_processed
x_test_noisy = add_gaussian_noise(x_test_processed, noise_factor)

print(f"Shape of x_train_noisy: {x_train_noisy.shape}")
print(f"Min value of x_train_noisy: {x_train_noisy.min()}, Max value of x_train_noisy:
{x_train_noisy.max()}")
print(f"Shape of x_test_noisy: {x_test_noisy.shape}")
print(f"Min value of x_test_noisy: {x_test_noisy.min()}, Max value of x_test_noisy:
{x_test_noisy.max()}")
```

Output :

```
Shape of x_train_noisy: (60000, 28, 28, 1)
Min value of x_train_noisy: 0.0, Max value of x_train_noisy: 1.0
Shape of x_test_noisy: (10000, 28, 28, 1)
Min value of x_test_noisy: 0.0, Max value of x_test_noisy: 1.0
```

Step 3 : Define autoencoder model

```
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, UpSampling2D
from tensorflow.keras import Model

# Input shape of the images (e.g., MNIST images are 28x28 with 1 channel)
input_shape = x_train_processed.shape[1:]

# 1. Input layer
```

```
input_img = Input(shape=input_shape)

# 2. Encoder part
x = Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
x = MaxPooling2D((2, 2), padding='same')(x)
x = Conv2D(64, (3, 3), activation='relu', padding='same')(x)
encoded = MaxPooling2D((2, 2), padding='same')(x)

# 3. Decoder part
x = Conv2D(64, (3, 3), activation='relu', padding='same')(encoded)
x = UpSampling2D((2, 2))(x)
x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
x = UpSampling2D((2, 2))(x)
decoded = Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)

# 4. Autoencoder model
autoencoder = Model(input_img, decoded)

# Print the model summary to verify the architecture
autoencoder.summary()
```

Output :

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 28, 28, 1)	0
conv2d (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_1 (Conv2D)	(None, 14, 14, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 64)	0
conv2d_2 (Conv2D)	(None, 7, 7, 64)	36,928
up_sampling2d (UpSampling2D)	(None, 14, 14, 64)	0
conv2d_3 (Conv2D)	(None, 14, 14, 32)	18,464
up_sampling2d_1 (UpSampling2D)	(None, 28, 28, 32)	0
conv2d_4 (Conv2D)	(None, 28, 28, 1)	289

Total params: 74,497 (291.00 KB)
Trainable params: 74,497 (291.00 KB)
Non-trainable params: 0 (0.00 B)

Step 4 : Train Autoencoder model

```
history = autoencoder.fit(
    x_train_noisy,
    x_train_processed,
    epochs=10,
    batch_size=128,
    shuffle=True,
    validation_data=(x_test_noisy, x_test_processed)
)

print("Autoencoder model training complete. Training history stored in 'history' variable.")
```

Output :

```
Epoch 1/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 108s 227ms/step - loss: 0.1233 - val_loss: 0.0816
Epoch 2/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 115s 245ms/step - loss: 0.0793 - val_loss: 0.0767
Epoch 3/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 144s 249ms/step - loss: 0.0764 - val_loss: 0.0750
Epoch 4/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 112s 240ms/step - loss: 0.0749 - val_loss: 0.0737
Epoch 5/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 102s 218ms/step - loss: 0.0740 - val_loss: 0.0730
Epoch 6/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 115s 244ms/step - loss: 0.0733 - val_loss: 0.0724
Epoch 7/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 143s 246ms/step - loss: 0.0727 - val_loss: 0.0721
Epoch 8/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 141s 244ms/step - loss: 0.0723 - val_loss: 0.0717
Epoch 9/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 143s 246ms/step - loss: 0.0720 - val_loss: 0.0713
Epoch 10/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 145s 251ms/step - loss: 0.0717 - val_loss: 0.0714
Autoencoder model training complete. Training history stored in 'history' variable.
```

Step 5 : Evaluate and Visualize the model

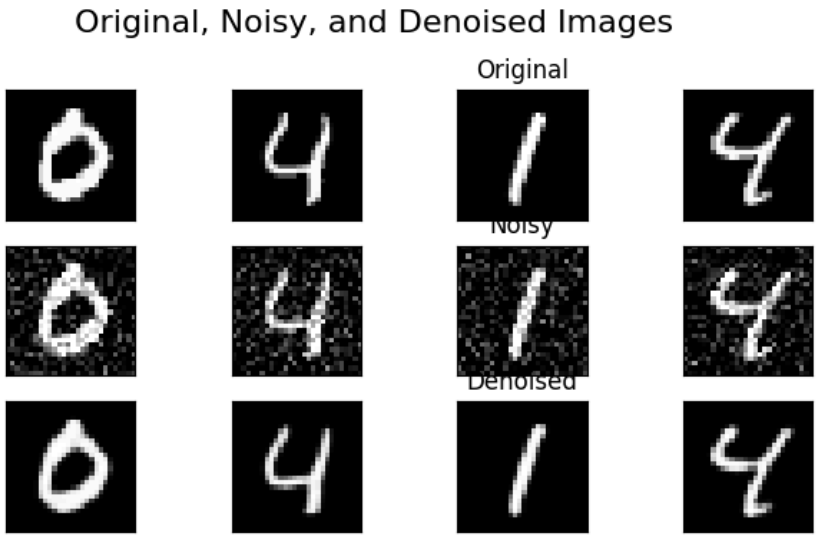
```
decoded_imgs = autoencoder.predict(x_test_noisy)
print("Denoised images predicted and stored in 'decoded_imgs'.")
print(f"Shape of decoded_imgs: {decoded_imgs.shape}")
```

Output :

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 5s 15ms/step
Denoised images predicted and stored in 'decoded_imgs'.
Shape of decoded_imgs: (10000, 28, 28, 1)
```

```
import matplotlib.pyplot as plt

# 3. Choose a few (e.g., 5-10) image indices to visualize from the test set.
n = 10  # Number of images to display
plt.figure(figsize=(20, 4))

for i in range(n):
    # Display Original Image
    ax = plt.subplot(3, n, i + 1)
    plt.imshow(x_test_processed[i].reshape(28, 28), cmap='gray')
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
    if i == n // 2:
        ax.set_title('Original')

    # Display Noisy Image
    ax = plt.subplot(3, n, i + 1 + n)
    plt.imshow(x_test_noisy[i].reshape(28, 28), cmap='gray')
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
    if i == n // 2:
        ax.set_title('Noisy')

    # Display Denoised Image
    ax = plt.subplot(3, n, i + 1 + 2 * n)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
    if i == n // 2:
        ax.set_title('Denoised')

plt.suptitle('Original, Noisy, and Denoised Images', y=1.02, fontsize=16)
plt.show()
```

Output :



Conclusions :

The Convolutional Autoencoder was successfully implemented and trained to perform image denoising on handwritten digits.

Key findings from the experiment include:

1. **Feature Learning:** The model successfully learned to identify the structural components of digits (the "signals") while filtering out the random Gaussian perturbations (the "noise").
2. **Visual Verification:** The final visualization showed three distinct rows: Original, Noisy, and Denoised. The "Denoised" row clearly showed that the digits were reconstructed with high clarity, confirming the model's effectiveness.
3. **Loss Convergence:** Over 10 epochs, the training and validation loss decreased steadily. The use of a large batch size (128) provided stable gradient updates, helping the model converge efficiently.
4. **Reconstruction Quality:** While the denoised images were much clearer than the noisy ones, they were slightly softer or "blurrier" than the originals. This is a characteristic of autoencoders, as the compression process in the bottleneck layer naturally leads to a small loss of high-frequency detail.